Serverless Blue-Green Deployment

Motivation

Why Serverless ?

- No server management: AWS handles provisioning, scaling, and maintenance.
- Pay-per-use: Only pay for actual execution time.
- Scalability: Auto-scales with concurrent requests.
- Quick deployment: Faster release cycles for small applications.

Why Blue-Green Deployment:

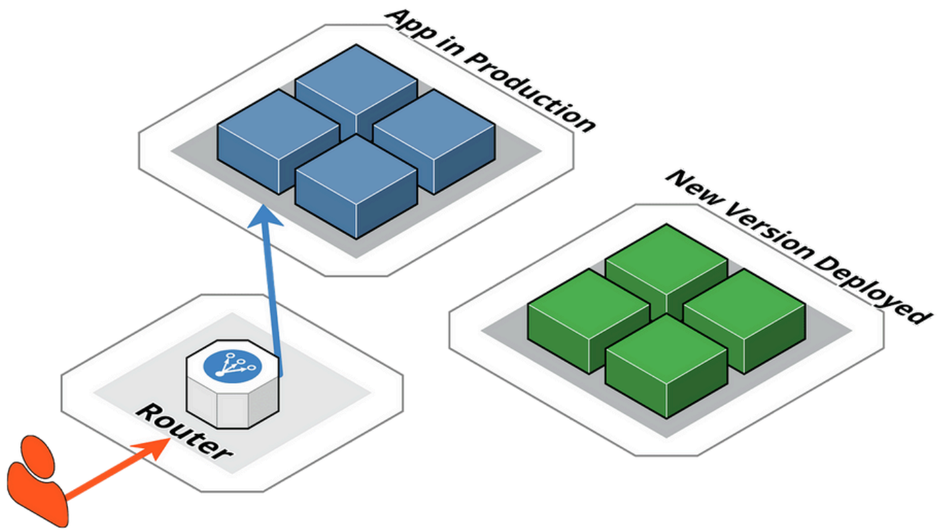
- Minimizes downtime by maintaining two identical environments.
- Allows instant rollback if new version fails.
- Reduces impact of deployment errors on end-users.
- Supports continuous delivery and frequent updates.

Blue-Green Deployment Architecture

What is Blue-green deployment:

Blue-Green Deployment is a release strategy that reduces downtime and risk by running two identical environments — Blue (current version) and Green (new version) — and switching traffic between them.

- Stable, live version currently serving users
- Updated version deployed separately for testing



Benefits:

- Zero-downtime updates
- Safe rollback if new version fails
- Test new version with real users before full rollout

Traffic switching :

- Start with 100% traffic to Blue
- Shift some % to Green (e.g., 10%)
- Gradually shift all traffic to Green if stable
- Rollback to Blue if issues occur

Implementation process

Version creation

Create separate Lambda function versions for Blue and Green by deploying changes and assigning Lambda aliases (Blue, Green) to maintain different code versions for each environment.

Green Development

Deployed the updated Green version (with small UI changes) using a new Lambda version and kept it at 0% traffic initially to allow safe and isolated testing.

Testing & Validation

Manually test Green version by calling its endpoints directly to verify that core functionalities (add/view/delete tasks) and UI updates worked correctly before traffic shifting.

Traffic migration

Used Lambda alias weighted traffic shifting to gradually move traffic from Blue to Green (starting at 10%) and monitored frontend behavior, then fully switched to Green once stable; rollback was tested by reverting alias if failure occurred.

Tech Stack and AWS Cloud Services:

Frontend: React

Backend: AWS Lambda (Node.js)

API Management: API Gateway

Authentication: Cognito

Database: DynamoDB

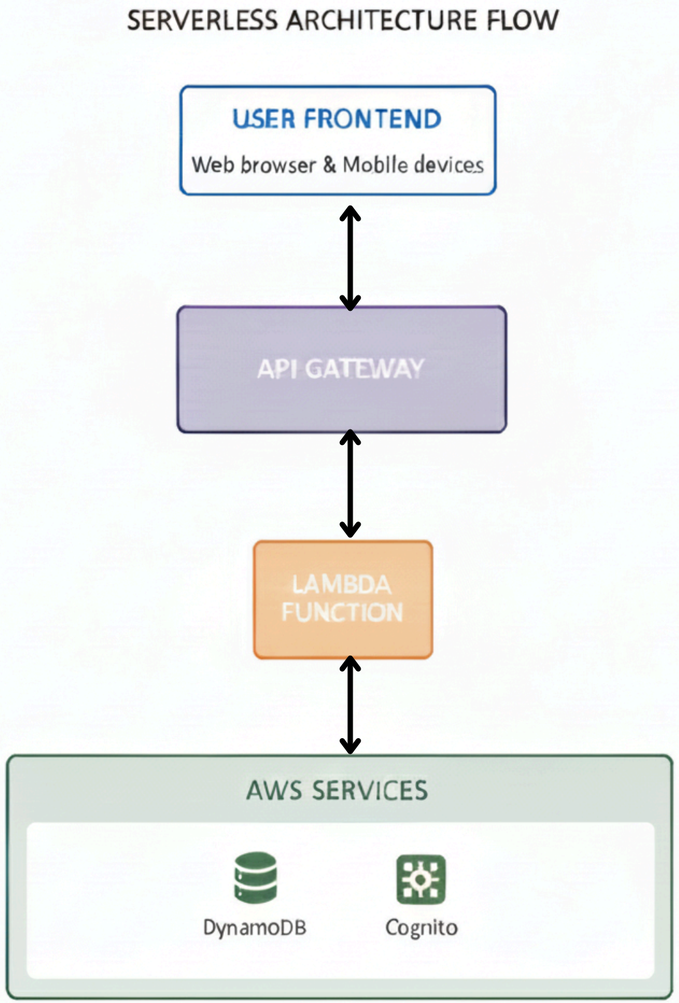
Implementation:

- **All application backend logic** (such as data management, user interactions, and workflow processing) is implemented using AWS Lambda, with each function triggered by specific user actions.
- **API Gateway** exposes RESTful endpoints, allowing the frontend to communicate securely and efficiently with Lambda functions.
- **DynamoDB** serves as the persistent and scalable data store for structured application data (e.g., tasks, user profiles, records).
- **Amplify** hosts the frontend and facilitates automated CI/CD deployments for fast updates.
- **Cognito** handles user authentication and authorization, ensuring secure access control across the application.

SystemFlow/Architecture :

- The user interacts with the app, like adding a task, viewing data, or submitting a request, which generates an HTTP request to the system.

- API Gateway receives this request and routes it to the appropriate Lambda function, which executes the business logic, interacts with AWS services like DynamoDB for data storage and sends the processed result back to the user



Contributions

Pardhuva	• Managing Lambda versions and aliases for Blue-Green environments. • Conduct testing, weighted traffic switching, monitoring, and rollback. • Connecting Cognito with API Gateway and frontend.
Vyshnavi	• Frontend development (UI, API integration) • Cognito setup & user management. • Dealing with Amplify, S3 and CloudFront.
Bhavishya	• Backend development (Lambda functions) • Dynamodb Data modelling • API Gateway endpoints and backend logic for tasks.

Thank You